练习：在 Pull Request 上运行 Cars CRUD 测试

目标：编写一个 GitHub Actions 工作流，每当有 pull request 以 main 为目标时，自动运行 Cars CRUD 应用的现有单元、集成、API 和端到端（E2E）测试。

前提条件：你已完成 GITHUB_ACTIONS_GUIDE.md 和 TESTING_WORKSHOP.md。
workshop 中的测试应当已经提交到 main（如之前的 GitHub 练习中那样）。

1. 准备一个分支

在 cars-crud/ 的根目录下：

```
git switch main
git pull --ff-only origin main
git switch -c github-actions-tests
```

如果 github-actions-tests 已存在，直接切换到它，而不要再次创建。

2. 要求

创建 .github/workflows/tests.yml。它必须：

1. 针对以 main 为目标的 pull request 运行，包括向该 PR 推送新提交时。不要在本练习中添加 push 触发器。
2. 使用 Ubuntu 运行器和 Python 3.12。
3. 将仓库文件下载到运行器。
4. 从 requirements-dev.txt 安装应用的测试依赖。
5. 安装 Playwright Chromium 及其 Linux 系统依赖。
6. 执行全部四个测试组，最好作为单独的命名步骤：
   - tests/unit/
   - tests/integration/
   - tests/api/
   - tests/e2e/
7. 如果任一测试失败则显示失败的运行，全部通过时显示成功的运行。

你不需要单独的 SQLite 服务，不需要自己启动 Uvicorn，也不需要配置认证。
提供的 fixture 会创建隔离的数据库，E2E fixture 会启动它自己的 FastAPI 服务器。

3. 起始大纲

使用下面的指南和提示填写缺失的部分。这是故意不完整的，不是
完整的工作流：

```
name: Cars CRUD Tests

# TODO：添加过滤到 main 分支的 pull_request 事件
```

```
permissions:
  contents: read

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      # TODO: 检出仓库
      # TODO: 设置 Python 3.12
      # TODO: 安装 requirements-dev.txt
      # TODO: 安装 Chromium 及其 Linux 依赖
      # TODO: 运行单元测试
      # TODO: 运行集成测试
      # TODO: 运行 API 测试
      # TODO: 运行 E2E 测试
```

4. 提示

提示 A：触发与分支。使用 pull_request 事件及其 branches 过滤器。这里 main 是 PR 的目标分支，而不是源分支 github-actions-tests。

提示 B：准备运行器。复用入门指南中的 uses: 步骤：

- actions/checkout@v7 用于访问代码。
- actions/setup-python@v7，python-version: "3.12"。

提示 C：安装 Python 依赖。项目有 requirements-dev.txt，其中也包含普通的应用依赖：

```
python -m pip install -r requirements-dev.txt
```

提示 D：准备浏览器。本项目使用 Chromium，而不是 Firefox 或 WebKit。在 Ubuntu 运行器上，安装 Chromium 以及必要的操作系统库：

```
python -m playwright install --with-deps chromium
```

提示 E：运行四个测试组。这些与你在测试 workshop 中使用的命令相同：

```
python -m pytest tests/unit -q
python -m pytest tests/integration -q
python -m pytest tests/api -q
python -m pytest tests/e2e -q
```

使用 - name: 和 run: 将每条所需的 shell 命令转换成 YAML 步骤。默认情况下，返回非零退出状态的命令会使该步骤失败，因此不要抑制测试错误或添加 continue-on-error.

5. 推送并打开 pull request

完成 YAML 文件后：

```
git add .github/workflows/tests.yml
git commit -m "Run Cars CRUD tests on pull requests"
git push -u origin github-actions-tests
```

在 GitHub 上：

1. 选择 Compare & pull request 或 Pull requests > New pull request。
2. 设置 base: main 和 compare: github-actions-tests。
3. 创建 pull request。
4. 打开其 Checks 部分（或 Actions 标签页），选择 Cars CRUD Tests 运行，并检查每个步骤的输出。
5. 确认单元、集成、API 和 E2E 测试全部通过。

当你向已打开的 pull request 推送更多提交时，工作流也应当再次运行。

6. 证明失败的测试会被检测到

1. 在 tests/unit/test_schemas.py 中，找到 test_color_is_optional。
2. 暂时将 assert car.color == "" 改为 assert car.color == "Blue"。
3. 提交并推送更改到 github-actions-tests。
4. 打开 PR 的 checks，找到失败的单元测试步骤。后面的步骤可能会被跳过，因为作业在失败处停止了。
5. 恢复 assert car.color == ""，提交并再次推送。
6. 验证最新的运行成功，然后将 PR 合并到 main。

不要在合并的代码中留下故意失败的断言。

7. 完成检查清单

- [ ] 我的工作流存储在 .github/workflows/tests.yml。
- [ ] 它只针对以 main 为目标的 pull request 触发。
- [ ] 它配置了 Ubuntu 和 Python 3.12。
- [ ] 它安装了 requirements-dev.txt 和 Playwright Chromium。
- [ ] 在 GitHub 运行中，四个测试组全部通过。
- [ ] 故意破坏一个单元测试会产生失败的检查。
- [ ] 修复测试后会产生一次成功的新检查。
- [ ] 完成的 pull request 被合并到 main。

故障排查

问题	提示
PR 上没有出现任何运行	确认文件路径、有效的 YAML、已启用的 Actions，以及 PR 的目标是 `main`.
`No module named pytest`	在运行测试之前安装 requirements-dev.txt。
Chromium 可执行文件缺失	在 E2E 测试之前运行 Playwright 安装步骤。
Linux 共享库错误	使用 python -m playwright install --with-deps chromium，而不是仅 install chromium。
浏览器无法访问应用	检查 E2E fixture 的输出。它会自动启动本地服务器；不需要单独的启动应用步骤。
后面的测试组没有运行	前面的步骤可能已失败；先检查该步骤的日志。

参考：Playwright Python：在 GitHub Actions 中运行。